

PA1 实验报告

林晖鹏 2312966 计算机学院

目录

1. PA0 报告
 - 1.1 使用的系统环境
 - 1.2 安装与使用的工具
 - 1.3 遇到的环境问题与解决过程
 - 1.4 NEMU 初次编译与运行结果
 - 1.5 对 PA0 的理解
2. 概述
3. 实验目的
4. 实验内容
5. 阶段一：NEMU 基础设施与调试器基本命令
 - 5.1 NEMU 启动与执行流程
 - 5.2 `CPU_state` 的寄存器组织
 - 5.3 NEMU 命令循环与执行框架的初步理解
 - 5.4 `si` 单步执行命令
 - 5.5 `info r` 寄存器打印命令
 - 5.6 `x` 内存扫描命令
 - 5.7 阶段一小结
6. 阶段二：表达式求值
 - 6.1 表达式词法分析
 - 6.2 表达式递归求值
 - 6.3 单目负号、减号、乘法和指针解引用
 - 6.4 `p` 表达式求值命令
 - 6.5 表达式求值与编译器前端的联系
 - 6.6 阶段二小结
7. 阶段三：监视点
 - 7.1 监视点池管理
 - 7.2 监视点触发机制
 - 7.3 软件断点与 `int3` 的一点理解
 - 7.4 阶段三小结

8. 必答题

- 8.1 i386 手册查阅位置
- 8.2 代码行数统计
- 8.3 `-Wall` 与 `-Werror`

9. 实验过程中遇到的问题与解决方法

- 9.1 `CPU_state` 组织不正确会导致 NEMU 一启动就失败
- 9.2 `si` 命令的参数处理一开始考虑得不完整
- 9.3 `x` 命令最开始没有把表达式和地址真正连接起来
- 9.4 表达式求值里最难处理的是单目负号和二元减号的区分
- 9.5 token 缓冲区越界问题不能靠“把数组开大一点”解决
- 9.6 监视点一开始的返回值设计不合理，导致总是被误判为触发

10. 总结

1. PA0 报告

PA0 虽然没有单独的必答题，但它直接决定了后续 PA1 和 PA2 能否顺利开展。PA0 的核心任务不是实现某个具体功能，而是把实验所需的 GNU/Linux 开发环境、编译工具链、调试工具链和工程目录组织好，并确保 NEMU 可以被正常编译和启动。只有这一部分稳定下来，后续对源码的修改、测试和调试才有可靠基础。

1.1 使用的系统环境

本次实验最终采用 GNU/Linux 开发环境完成。PA 的实验环境要求本质上比较明确：

- 能够使用 shell 进入工程目录并执行命令；
- 能够使用 GCC 正常编译 C 工程；
- 能够使用 GNU Make 构建 NEMU；
- 能够使用 GDB 或日志输出来辅助调试；
- 能够正确管理实验代码目录、镜像文件和相关依赖。

在实际使用中，PA0 并不只是“把系统装好”，而是要把后续实验反复会用到的工作方式先固定下来，例如：

- 在终端中切换目录；
- 查看和编辑源文件；
- 使用 `make` 构建工程；
- 使用 `git` 管理代码；

- 使用 **gdb**、日志文件和 shell 命令定位问题。

1.2 安装与使用的工具

PA0 阶段完成配置并实际使用到的工具主要包括：

- GCC
- GNU Make
- GDB
- Git
- shell 常用命令
- 文本编辑器
- 课程实验所需依赖环境

其中，各工具在实验中的作用可以进一步细化如下：

- **gcc** 用于编译 NEMU 以及后续实验中的测试程序。没有编译器，后续所有源码修改都无法验证。
- **make** 用于按 Makefile 中的规则组织编译过程。PA 的整个工程都是围绕 Makefile 驱动的，因此 **make** 是最常用的入口命令之一。
- **gdb** 用于查看程序运行状态、设置断点、单步调试和观察寄存器/内存变化。即使在 PA1 中主要实现的是 NEMU 内部的简易调试器，外部调试工具仍然非常重要。
- **git** 用于获取实验代码、切换分支、比较修改前后差异，以及统计后续实验中新增的代码量。
- shell 常用命令 例如 **cd**、**ls**、**pwd**、**cp**、**mv**、**cat**、**find**、**grep**、**wc**、**which**、**man** 等。这些命令虽然看起来基础，但在环境配置、排错和统计信息时都非常高频。
- 文本编辑器 用于查看和修改源码、Makefile 以及实验记录文件。PA 的实验内容涉及大量源码阅读，因此编辑器使用效率会直接影响后续进度。
- 课程实验所需依赖环境 参考《计算机系统设计-准备工作.pdf》和《NK 实验 PA 流程.pdf》，这部分不是一个抽象概念，而是需要按讲义明确安装的一组环境和软件包。

结合准备工作文档，PA0 阶段应重点完成以下环境配置：

- 虚拟机环境 文档建议使用 **VMWare + Ubuntu** 的实验环境（so i do），教程示例中给出的镜像版本为 **ubuntu-16.04.4-desktop-i386.iso**，并强调创建虚拟机时选择 **Ubuntu 32 位** 系统。这样做的目的，是尽量统一实验平台，减少因为宿主机和系统版本差异带来的额外问题。

- 基础编译工具链 文档要求安装：

```
sudo apt-get install build-essential
```

该软件包实际包含了 GCC、G++、Make 以及常见构建工具，是后续编译 NEMU 的基础。

- 调试工具 文档要求安装：

```
sudo apt-get install gdb
```

用于后续调试 NEMU、观察程序执行过程、理解断点与单步执行等问题。

- 版本管理工具 文档要求安装：

```
sudo apt-get install git
```

用于获取实验源码、切换分支、查看历史版本、比较代码改动，也是完成后续代码统计和版本回溯的基础。

- 命令行交互库 文档要求安装：

```
sudo apt-get install libreadline-dev
```

NEMU 的 monitor 命令行界面依赖 **readline**，如果缺少这个开发库，工程在编译相关模块时就可能失败。

- 图形与输入输出相关库 文档要求安装：

```
sudo apt-get install libsdl2-dev
```

该库主要服务于后续 PA 中的设备模拟、图形显示和输入输出部分。虽然在 PA1 中未必立即全部用到，但环境阶段必须提前安装好，否则后续编译会缺依赖。

- 辅助仿真工具 文档要求安装：

```
sudo apt-get install qemu-system-x86
```

这部分工具与后续实验中的参考运行环境和比对过程有关，准备阶段同样要求提前安装。

```
sudo apt-get install build-essential
sudo apt-get install gdb
sudo apt-get install git
sudo apt-get install libreadline-dev
sudo apt-get install libsdl2-dev
sudo apt-get install qemu-system-x86
```

从后续实验的实际情况看，最关键的并不是“装了多少工具”，而是这些工具和依赖能否被稳定调用，并且命令行工作流是否已经顺手。尤其是 `build-essential`、`libreadline-dev` 和 `libsdl2-dev` 这一类软件包，如果在 PA0 阶段没有配置完整，后面往往不是“功能做不出来”，而是连工程都无法顺利编译。

1.3 遇到的环境问题与解决过程

本次实验在 PA0 阶段主要遇到的问题集中在以下几类：

- 路径配置不正确导致找不到命令或文件
- 依赖工具未安装完整导致构建失败
- 文件传输或目录结构错误导致工程不能正确编译
- 对 shell 命令不熟悉导致排错效率较低

这些问题在实验中的具体表现通常包括：

- 进入工程目录后执行命令，提示找不到文件或目录。这类问题通常与当前工作目录不对、文件路径写错或工程没有放到预期位置有关。解决方法是先用 `pwd` 和 `ls` 确认当前目录，再逐级检查工程目录结构是否完整。
- 执行 `make` 时出现缺少编译器、缺少依赖库或头文件找不到的报错。这类问题说明工具链没有配置完整，不能只盯着第一行错误信息看，而应该继续往下看具体缺的是哪一个命令、哪一个库或哪一个头文件，再逐项补齐。
- 代码目录、镜像文件或实验材料放置位置不规范，导致命令虽然输入正确，但工程实际上读不到对应文件。这类问题表面上像“编译失败”或“运行失败”，本质上是文件组织没有整理清楚，因此需要先把目录结构整理稳定，再进行后续实验。

- 对 shell 命令不够熟悉，出现“知道要看什么，但不知道怎么查”的情况。例如想统计代码行数、查找某个文件、查看命令帮助时，如果不会用 `find`、`wc`、`man`、`grep` 等命令，排错效率会明显下降。这部分问题只能通过边查边用逐渐解决（后续引入了让Cil作为一个代码查阅器，方便多啦。）

PA0 阶段虽然主要是环境搭建，但真正训练的是一种非常基础的能力：遇到问题时先检查环境、路径、命令、依赖和目录，而不是上来就怀疑代码本身有错。

1.4 NEMU 初次编译与运行结果

PA0 阶段最关键的结果，是在本地环境中完成 NEMU 的初次编译和启动。整个过程通常可以概括为：

1. 进入 `nemu/` 工程目录；
2. 执行 `make` 检查工程能否正常构建；
3. 执行 `make run` 或直接运行生成的可执行文件；
4. 观察程序是否能够进入 monitor。

如果 `make` 能顺利通过，且执行后能够进入：

```
Welcome to NEMU!  
For help, type "help"  
(nemu)
```

就说明最基础的实验环境已经具备。这个结果虽然看似简单，但实际上已经验证了几件关键事情：

- 编译器和 Makefile 能协同工作；
- 工程目录结构是完整的；
- NEMU 的依赖环境已经满足；
- 当前系统能够正确运行实验工程。

换句话说，这一步虽然还没有真正开始改代码，但它已经把“后续代码能否被验证”这个前提问题先解决了。

1.5 对 PA0 的理解

PA0 很容易被看成“正式实验开始前的准备工作”，但真正完成之后可以发现，它本身就是实验的一部分。PA0 不强调复杂原理，而是要求先建立一个稳定、可重复、可调试的实验环境。后续 PA1 和 PA2 中的很多效率问题，往往并不来自代码逻辑本身，而是来自环境、路径、工具链和命令行工作流是否稳定。正因为如此，PA0 做得是否扎实，会直接影响后续每一步实验的推进速度。

2. 概述

PA1 的主题是“最简单的计算机”。这一阶段的目标并不是直接实现一个复杂系统，而是先在 NEMU 中搭建最基本的运行和调试基础设施。具体来说，需要完成正确的寄存器组织、理解 NEMU 的启动与执行流程、补齐 monitor 的基础命令、实现表达式求值，并在此基础上实现监视点功能。虽然这些功能从用户角度看只是几个命令，但它们背后实际上把 CPU 状态、内存访问、字符串解析和程序执行控制连接了起来。

从实验过程来看，PA1 的意义主要体现在两个层面。第一，它使“程序如何运行”和“调试器如何工作”不再停留在抽象描述上，而是落实到具体代码和具体状态变化上。第二，它为 PA2 中更复杂的译码、执行和设备模拟准备了最关键的基础设施。如果 PA1 中的寄存器结构、表达式求值或 monitor 命令设计不稳，后续实验就会反复遇到难以定位的问题。

3. 实验目的

- 熟悉 GNU/Linux 环境下阅读、编译和调试工程代码的基本方式。
- 理解 TRM 的基本组成，包括内存、PC、寄存器和指令执行循环。
- 实现并理解简易调试器中的 `si`、`info r`、`x`、`p`、`w`、`d` 等命令。
- 实现表达式词法分析和递归求值过程。
- 实现监视点池和监视点触发机制。
- 学会根据问题快速查阅 i386 手册中的相关章节。

4. 实验内容

PA1 的实验内容可以分为三个阶段：

1. 阅读框架代码，理解 NEMU 的启动和客户程序执行流程，完成 `CPU_state` 寄存器结构体，以及 `si`、`info r`、`x` 等基础调试命令。

2. 实现表达式词法分析和递归求值，支持十进制、十六进制、寄存器、括号、单目负号、指针解引用以及常见运算符。
3. 实现监视点池管理和监视点触发机制，并结合断点、GDB、i386 手册和编译器相关知识完成总结与必答题。

5. 阶段一：NEMU 基础设施与调试器基本命令

5.1 NEMU 启动与执行流程

在开始补功能之前，先对 NEMU 的主流程进行了梳理。启动入口位于 `nemu/src/monitor/monitor.c` 的 `init_monitor()`，其主要流程如下：

```
parse_args()
init_log()
reg_test()
load_img()
restart()
init_regex()
init_wp_pool()
init_device()
welcome()
```

这些步骤分别对应命令行参数解析、日志初始化、寄存器结构检查、镜像加载、CPU 状态重置、表达式正则初始化、监视点池初始化、设备初始化和欢迎信息输出。通过这一过程可以看出，NEMU 的 monitor 并不是“先运行程序再调试”，而是先搭建完整的调试和运行环境，再等待用户用命令去驱动客户程序执行。

如果没有显式提供外部镜像，`load_default_img()` 会把默认客户程序加载到固定位置，随后 `restart()` 会设置入口地址。因此 PA1 阶段中，用户在 monitor 中输入 `c` 或 `si` 时，实际就是从默认入口地址开始驱动程序执行。

从指导书角度看，这部分内容对应的是“先理解程序是如何启动的，再实现调试基础设施”。如果一上来只盯着某个命令实现，很容易忽略 monitor、CPU 状态和客户程序之间的关系；而先把启动链路看清楚，后面实现 `si`、`info r` 和 `x` 时思路会更稳定。

5.2 CPU_state 的寄存器组织

x86 的通用寄存器并不是完全独立的 32 位变量，而是存在明显的位宽重叠关系：

```
32 位: EAX ECX EDX EBX ESP EBP ESI EDI
16 位: AX  CX  DX  BX  SP  BP  SI  DI
8 位:  AL  CL  DL  BL  AH  CH  DH  BH
```

例如：

- **AX** 是 **EAX** 的低 16 位。
- **AL** 是 **AX** 的低 8 位。
- **AH** 是 **AX** 的高 8 位。

这意味着 **CPU_state** 不能简单写成若干完全独立的字段，否则写入 **EAX** 后再读取 **AX** 或 **AL** 时，数据就不会自动保持一致。为了解决这个问题，在 **nemu/include/cpu/reg.h** 中使用了 **union** 来共享底层存储。核心结构如下：

```
union {
    union {
        uint32_t _32;
        uint16_t _16;
        uint8_t _8[2];
    } gpr[8];

    struct {
        rtlreg_t eax, ecx, edx, ebx, esp, ebp, esi, edi;
    };
};
```

union 的关键特性是：所有成员共享同一段起始地址相同的内存空间。也就是说，在同一个 **union** 中对 **_32** 写入一个 32 位值之后，再去读取 **_16** 或 **_8**，得到的不是另一份独立保存的数据，而是同一份二进制内容在不同位宽下的解释结果。这与 x86 通用寄存器的真实组织方式是一致的。**EAX**、**AX**、**AH**、**AL** 并不是四个互不相关的寄存器，而是同一寄存器内容在不同位宽和不同位置上的视图。

如果改用普通 **struct** 来分别保存 32 位、16 位和 8 位值，那么每次写 **EAX** 之后都需要再手动同步 **AX**、**AH**、**AL**；反过来写 **AL** 时还要反向更新 **AX** 和 **EAX**。这种做法不仅繁琐，而且极易出错。**union** 在这里并不是“语法技巧”，而是把这种本该手动维护的一致性直接交给底层共享内存来保证。

这样设计之后，可以同时支持两类访问方式：

```
reg_l(index)
reg_w(index)
reg_b(index)
cpu.eax
cpu.ecx
```

其中 `reg_b(index)` 的实现尤其体现了 x86 8 位寄存器编码的特点：

```
#define reg_b(index) (cpu.gpr[check_reg_index(index) & 0x3]._8[index >> 2])
```

这里：

- `index & 0x3` 用于选择前四个通用寄存器之一。
- `index >> 2` 用于区分低 8 位和高 8 位。

原因在于 x86 的 8 位寄存器只有：

```
AL CL DL BL AH CH DH BH
```

也就是说，8 位寄存器只来自前四个 32 位寄存器，而不是全部八个通用寄存器。

这个部分和指导书中“正确实现寄存器结构体”的要求是完全对应的。指导书强调要先通过 `reg_test()`，本质上就是要求寄存器的物理重叠关系必须先成立，因为后续所有指令执行和调试输出都建立在这个结构之上。如果这个结构错了，NEMU 一开始就会在 `reg_test()` 中失败，后面所有功能都无从谈起。

```
[src/monitor/monitor.c,50,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 05:46:12, Apr 22 2026
For help, type "help"
(nemu) █
```

5.3 NEMU 命令循环与执行框架的初步理解

NEMU 的命令行交互位于 `nemu/src/monitor/debug/ui.c` 的 `ui_mainloop()` 中。这个循环通过 `readline()` 读入用户输入，再从 `cmd_table` 中查找对应命令并调用其处理函数。也就是说，monitor 层负责做的事情其实只有两件：

1. 把用户输入解析成命令和参数。
2. 调用对应的处理函数。

真正的客户程序执行则由 `cpu_exec()` 驱动，它内部会反复调用执行框架去跑客户指令。因此从职责分工上看：

- `ui_mainloop()` 解决“用户想做什么”。
- `cpu_exec()` 解决“客户程序怎样执行”。

这个认识很重要，因为它解释了为什么实现 `si` 时不需要重写取指、译码、执行逻辑，为什么实现 `info r` 时只需要读取 `cpu` 结构，为什么实现 `x` 时只需要“先算地址，再读内存”。

5.4 `si` 单步执行命令

`si` 命令的实现位置在 `nemu/src/monitor/debug/ui.c`。核心代码如下：

```
static int cmd_si(char *args) {
    uint64_t n = 1;

    if (args != NULL) {
        n = strtoull(args, NULL, 10);
        if (n == 0) {
            printf("Usage: si [N], and N should be greater than 0\n");
            return 0;
        }
    }

    cpu_exec(n);
    return 0;
}
```

指导书要求在 PA1 中实现简易调试器的基础功能，`si` 正是其中最核心的一项。它的作用是让客户程序按指令为单位逐步执行，而不是一次性连续运行到底。之所以必须先实现这个命令，是因为后续很多调试行为都依赖“执行一条指令后立刻停下”这一能力。只有程序能够在每条指令之间暂停，寄存器、内存和程序计数器的变化才可以被精确观察。

如果只有 `c` 命令而没有 `si`，程序会一直运行到结束或出错，中间过程几乎无法细看；而 `si` 可以把执行过程细化到“每一条客户指令”，这样就能在执行后立即结合 `info r`、`x` 等命令检查状态变化，判断当前实现是否符合预期。从这个意义上说，`si` 并不只是一个“方便测试的小命令”，而是简易调试器最基础的步进执行能力。

这段代码背后的设计点有三个：

1. `si` 默认执行 1 条指令，因此先把 `n` 初始化为 1。

2. 用户如果给出参数，例如 `si 3`，就将参数解析成无符号整数。
3. 参数为 `0` 时没有调试意义，所以直接提示并返回。

`cmd_si()` 本身并不执行指令，它只是把用户输入转换成 `cpu_exec(n)` 所需要的参数。真正执行客户程序的仍然是：

```
cpu_exec(n);
```

这样实现的好处是，monitor 层不会重复 CPU 层已经具备的能力。`si` 只是一个前端命令接口，而 `cpu_exec()` 才是执行引擎。

从指导书要求来看，`si` 的实现体现了“理解框架优先于重写框架”。指导书希望补全调试器命令，但并不鼓励重复造轮子。只要把 monitor 命令参数正确转交给已有的执行框架，就已经是符合系统结构的实现。

```
[src/monitor/monitor.c,50,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 05:46:12, Apr 22 2026
For help, type "help"
(nemu) si
      100000:  b8 34 12 00 00          movl $0x1234,%eax
Watchpoints were triggered.
(nemu) si 20
Watchpoints were triggered.
(nemu) █
```

5.5 `info r` 寄存器打印命令

`info r` 也是在 `nemu/src/monitor/debug/ui.c` 中实现的。核心辅助函数如下：

```
static void print_reg() {
    int i;
    for (i = R_EAX; i <= R_EDI; i++) {
        printf("%s\t0x%08x\t%u\n", reg_name(i, 4), reg_l(i), reg_l(i));
    }
    printf("eip\t0x%08x\t%u\n", cpu.eip, cpu.eip);
}
```

指导书中把这一功能称为“打印程序状态”。在实际调试中，仅仅让程序能单步执行还不够，还必须能够在停下之后看到 CPU 当前处于什么状态。寄存器是程序执行过程中最直接、最核心的一组状态信息，例如算术结果、地址计算结果、栈顶位置以及下一条将要执行的指令位置，都需要通过寄存器来观察。因此，`info r` 的意义就在于把“程序停下来了”和“程序现在是什么状态”连接起来。

尤其是在 PA1 当前阶段，最值得关注的是 8 个 32 位通用寄存器和 `eip`。执行 `si` 之后，如果没有 `info r`，就无法确认指令究竟改动了哪些寄存器，也无法判断 `eip` 是否正确前进；而有了 `info r`，就可以直接验证当前执行结果是否与反汇编语义一致。因此，`info r` 与 `si` 实际上是配套出现的：前者负责“看状态”，后者负责“推动状态变化”。

这里的实现思路很清楚：

- 8 个 32 位通用寄存器通过 `reg_name(i, 4)` 和 `reg_l(i)` 读取。
- `eip` 不属于 `gpr[8]`，所以单独打印。
- 同时输出十六进制和十进制值，兼顾地址观察和普通数值分析。

随后在 `cmd_info()` 中加入 `r` 子命令分支：

```
else if (strcmp(args, "r") == 0) {
    print_reg();
}
```

之所以选择打印 8 个 32 位寄存器和 `eip`，而不是把 8 位和 16 位寄存器也全部列出，是因为在 PA1 当前阶段，这些信息已经足够支持调试，而且不会让输出过于冗长。尤其是 `eip`，它直接反映当前或下一条指令的位置，在单步调试时非常重要。

从指导书要求看，`info r` 对应的是“打印程序状态”。这里的重点不是把所有底层状态都毫无筛选地输出，而是选出当前阶段最关键的一组状态，使它既能证明实现正确，也方便后续调试。

```

(nemu) info r
eax      0x4d04874d      1292142413
ecx      0x7d099659      2097780313
edx      0x697a1180      1769607552
ebx      0x108d79c9      277707209
esp      0x6f3e23ae      1866343342
ebp      0x601833b3      1612198835
esi      0x05aea451      95331409
edi      0x6a1fc00a      1780465674
eip      0x00100000      1048576
(nemu) si
100000:  b8 34 12 00 00          movl $0x1234,%eax
Watchpoints were triggered.
(nemu) info r
eax      0x00001234      4660
ecx      0x7d099659      2097780313
edx      0x697a1180      1769607552
ebx      0x108d79c9      277707209
esp      0x6f3e23ae      1866343342
ebp      0x601833b3      1612198835
esi      0x05aea451      95331409
edi      0x6a1fc00a      1780465674
eip      0x00100005      1048581
(nemu)

```

5.6 x 内存扫描命令

x 命令的格式为：

```
x N EXPR
```

其中：

- **N** 表示读取多少个 4 字节内存单元。
- **EXPR** 表示起始地址表达式。

指导书要求实现“扫描内存”功能，其目的在于让调试器不仅能看到寄存器状态，还能进一步观察某段内存中的实际内容。在程序调试过程中，很多关键信息并不只保存在寄存器里，例如栈中的返回地址、数据区中的变量值、某个地址开始的指令编码等，都需要通过内存扫描来检查。因此，x 命令的作用就是把“地址”转换成“可读的内存内容”，从而补全调试器的观察能力。

与 **info r** 相比，x 更进一步，因为它依赖表达式求值结果作为起始地址。也就是说，调试者不一定只能输入一个固定常数地址，还可以输入寄存器加偏移、十六进制常量等表达式。这使得 x 命令更接近真实调试器的使用方式。例如在查看栈内容时，可以直接使用 **x 10 \$esp**，而不必先手动算出 **\$esp** 当前的数值。

核心实现逻辑如下：

```
bool success = false;
vaddr_t addr = expr(expr_str, &success);
if (!success) {
    printf("Bad expression: %s\n", expr_str);
    return 0;
}

for (i = 0; i < n; i++) {
    if (i % 4 == 0) {
        printf("0x%08x:", addr + i * 4);

        printf("\t0x%08x", vaddr_read(addr + i * 4, 4));

        if (i % 4 == 3 || i == n - 1) {
            printf("\n");
        }
    }
}
```

这里有两个关键点。

第一，`x` 命令不是直接接收一个整数地址，而是接收一个表达式地址。这意味着它天然依赖表达式求值模块，因此也正好把 `monitor` 命令和表达式系统连接在了一起。

第二，最终内存读取统一通过：

```
vaddr_read(addr + i * 4, 4)
```

来完成，说明 `monitor` 不直接越过 NEMU 的内存接口，而是复用已有的访存路径。这样做更符合系统结构，也避免后续实验中因接口不统一带来额外问题。

为了让 `x 4 0x100000` 这种更自然的写法可以工作，表达式系统中还补充了十六进制常量支持。否则用户就只能写十进制地址，这和调试器的常见使用习惯明显不一致。

从指导书角度看，这一步对应的是“扫描内存”功能，但更深层的理解是：调试器本身也必须尊重模拟器已有的数据通路，而不是自己偷偷维护另一套内存读取逻辑。

```
[src/monitor/monitor.c,32,welcome] Build time: 05:46:12, Apr 22 2026
For help, type "help"
(nemu) x 4 0x100000
[src/monitor/debug/expr.c,124,make_token] match rules[18] = "[0-9a-fA-F]+" at position 0 with len 8: 0x100000
0x00100000: 0x001234b8 0x0027b900 0x01890010 0x0441c766
(nemu) help x
x - Examine memory: x N EXPR
(nemu) x 6 1048576
[src/monitor/debug/expr.c,124,make_token] match rules[19] = "[0-9]+" at position 0 with len 7: 1048576
0x00100000: 0x001234b8 0x0027b900 0x01890010 0x0441c766
0x00100010: 0x02bb0001 0x66000000
(nemu)
```


5.7 阶段一小结

这一阶段对应的关键代码主要集中在 `reg.h` 与 `ui.c` 中。`CPU_state` 和 `reg_test()` 之间是一一对应的关系：`reg_test()` 并不是随意写的测试代码，而是在验证寄存器结构是否真的满足 x86 的位宽重叠关系。`si`、`info r`、`x` 三个命令虽然都属于 monitor 命令，但接入系统的方式并不相同：`si` 把参数交给 `cpu_exec(n)`，`info r` 直接读取 `CPU_state`，`x` 则先调用 `expr()` 算地址，再通过 `vaddr_read()` 读内存。这说明 monitor 命令不应该绕开已有框架，而应尽量复用底层接口。

阶段一中暴露出的主要问题也比较典型：`CPU_state` 结构如果组织错误，会直接在启动时触发 `reg_test()` 失败；`si` 命令如果不处理缺省参数和非法参数，就无法同时支持 `si` 与 `si N` 两种形式；`info r` 如果不把 `eip` 一并打印出来，单步调试时就无法直观看到执行位置变化；`x` 命令如果不支持十六进制地址，使用体验就会明显偏离调试器常见习惯。

从测试结果来看，阶段一的实现已经形成闭环。执行 `make` 可以正常通过，说明修改没有破坏构建流程；运行 `make run` 能进入 `(nemu)` 提示符，说明 monitor 启动正常；测试 `help si`、`si`、`si 3` 可以验证命令注册与执行条数控制正确；连续执行 `info r`、`si`、`info r` 可以观察到 `eax` 和 `eip` 的变化与反汇编结果一致；执行 `x 4 0x100000` 与 `x 6 1048576` 都能正确读出内存内容，说明表达式地址与访存接口已经成功衔接。

阶段一完成后，NEMU 已经具备了最基础的调试能力。就实现内容来看，这一阶段完成了寄存器结构体、单步执行、寄存器打印和内存扫描；就理解层面来看，monitor、CPU 状态和客户程序执行这三层关系也逐渐清晰了。这个区分非常重要，因为它决定了后续表达式求值和监视点应该接在系统的什么位置上，而不只是“把功能做出来”。

6. 阶段二：表达式求值

6.1 表达式词法分析

表达式词法分析位于 `nemu/src/monitor/debug/expr.c`。其目标是把用户输入的字符串切成 token。例如：

```
1 + 2 * $eax
```


会被切分为数字、运算符和寄存器等 token，后续递归求值阶段只需要处理 token 数组即可。

这里使用 POSIX 正则表达式来描述 token 规则，例如：

```
"0[xX][0-9a-fA-F]+", TK_NUM}
{"[0-9]+", TK_NUM}
{"\\$(eip|eax|ecx|edx|ebx|esp|ebp|esi|edi)", TK_REG}
```

规则顺序非常重要。比如：

- `!=` 必须放在 `!` 前面。
- `==` 必须整体识别。
- 十六进制数字规则要放在十进制数字前面。

否则像 `0x100000` 这种输入，就可能先被错误识别成 `0`。

为了避免 token 数组越界，这里没有简单粗暴地把数组变大，而是写了统一的追加接口，在写入 token 前先做边界检查：

```
#define MAX_TOKENS 32

static bool append_token(int type, const char *str, int len) {
    if (nr_token >= MAX_TOKENS) {
        printf("Error: too many tokens, maximum is %d\n", MAX_TOKENS);
        return false;
    }

    if (len >= (int)sizeof(tokens[nr_token].str)) {
        printf("Error: token too long: %.*s\n", len, str);
        return false;
    }
    ...
}
```

这样设计的意义在于：越界问题不应该通过“侥幸扩大数组”来掩盖，而应该在每次写入前就被显式拦住。否则 monitor 里出现数组越界，后果往往不是简单的报错，而是非常难定位的随机行为。

词法分析中另一个关键点是处理 `-` 和 `*` 的二义性：

- `-` 既可能是二元减法，也可能是单目负号。
- `*` 既可能是乘法，也可能是指针解引用。


```
(1 + 2) * 3
```

真正的主运算符应当是最外层的 $*$ ，而不是括号中的 $+$ 。因此在扫描 token 区间时，需要维护括号层数，只有在最外层看到的运算符才可能成为当前层的主运算符。

优先级处理通过 `get_priority()` 完成，优先级数字越小，越适合作为当前层的主运算符。例如：

```
1 + 2 * 3
```

最外层主运算符应当是 $+$ ，而不是 $*$ ，因为乘法优先级更高，应在下一层递归中处理。

对于同优先级的二元运算符，这里选择最右侧运算符作为主运算符，从而保持左结合。例如：

```
1 - 2 - 3
```

会按：

```
(1 - 2) - 3
```

来计算。

这部分和指导书中“表达式求值”的要求是直接对应的，但更重要的意义在于：它把“先把字符串结构化，再按结构递归处理”的过程完整地体现了出来，这与编译器前端中的词法分析、语法分析思想是相通的。

在实际实现中，递归求值并不是凭空“不断调用自己”，而是建立在 `check_parentheses()`、`get_priority()`、`find_operator()` 这几个辅助函数之上的。真实代码中的关键部分如下：

```
int get_priority(int type) {
    switch (type) {
        case TK_OR:   return 1;
        case TK_AND:  return 2;
        case '|':      return 3;
        case '^':      return 4;
        case '&':      return 5;
```

```

    case TK_EQ:
    case TK_NE: return 6;
    case TK_LT:
    case TK_LE:
    case TK_GT:
    case TK_GE: return 7;
    case TK_SHL:
    case TK_SHR: return 8;
    case '+':
    case '-': return 9;
    case '*':
    case '/':
    case '%': return 10;
    default: return 1000;
}
}

```

```

int find_operator(int p, int q) {
    int pos = -1;
    int priority = 1000;
    for (int i = p; i <= q; i++) {
        if (tokens[i].type == TK_NUM || tokens[i].type == TK_REG) {
            continue;
        }
        if (tokens[i].type == TK_NEG || tokens[i].type == TK_NOT ||
            tokens[i].type == TK_BNOT || tokens[i].type == TK_DEREF) {
            continue;
        }
        else if (tokens[i].type == '(') {
            int cnt = 0;
            for (int j = i; j <= q; j++) {
                if (tokens[j].type == '(') cnt++;
                else if (tokens[j].type == ')') cnt--;
                if (cnt == 0) {
                    i = j;
                    break;
                }
            }
        }
        else {
            int current_priority = get_priority(tokens[i].type);
            if (current_priority <= priority) {
                priority = current_priority;
                pos = i;
            }
        }
    }
    return pos;
}

```

```

uint32_t eval(int p, int q, bool *success) {
    if (p > q) {
        *success = false;
        return 0;
    }
    else if (p == q) {
        if (tokens[p].type == TK_NUM) {

```

```

        return strtoul(tokens[p].str, NULL, 0);
    }
    if (tokens[p].type == TK_REG) {
        return reg_value(tokens[p].str, success);
    }
    *success = false;
    return 0;
}
else if (check_parentheses(p, q) == true) {
    return eval(p + 1, q - 1, success);
}
else {
    int op = find_operator(p, q);
    ...
}
}

```

这段代码体现了表达式递归求值的几个关键环节：

1. 先处理边界情况，例如空区间和单个 token。
2. 如果最外层是一对完整括号，就先去掉括号。
3. 否则寻找当前层真正起支配作用的主运算符。
4. 以主运算符为界，把表达式拆成左右两部分递归计算。

其中最关键的部分是主运算符选择。因为表达式并不是简单地“从左到右算”，而是要同时考虑括号层次、运算符优先级和结合性。在当前实现中，`get_priority()` 明确把优先级编码成数字，并且约定“数字越小，优先级越低，越适合作为当前层主运算符”。比如：

1 + 2 * 3

如果错误地把 * 当成最外层主运算符，就会把表达式拆成：

(1 + 2) * 3

显然结果会变错。因此，代码里的 `find_operator()` 本质上就是在做下面三件事：

- 跳过括号内部的运算符；
- 在最外层运算符中寻找优先级最低的一个；
- 对于同优先级运算符，按照左结合规则选择最右侧那个。

例如表达式：

1 - 2 - 3

真正的计算顺序应当是：

(1 - 2) - 3

因此主运算符应当选第二个 `-`，而不是第一个 `-`。这也正是代码里：

```
if (current_priority <= priority) {
    priority = current_priority;
    pos = i;
}
```

要写成 `<=` 而不是 `<` 的原因。使用 `<=` 时，同优先级下后出现的运算符会覆盖前一个位置，于是最终保留下来的就是最右侧运算符，从而实现左结合。

括号检查同样非常关键。当前实现中的 `check_parentheses()` 不是只检查两端是否是 `'('` 和 `')'`，还会继续扫描区间内部的括号层次。以：

(1 + 2) * 3

为例，虽然整个区间中最先出现的运算符是括号里的 `+`，但它不能作为最外层主运算符，因为它处在更深一层的括号内。也正因为如此，`find_operator()` 在扫描到 `'('` 时，不是继续逐个向后看，而是直接把整个括号子表达式跳过去：

```
else if (tokens[i].type == '(') {
    int cnt = 0;
    for (int j = i; j <= q; j++) {
        if (tokens[j].type == '(') cnt++;
        else if (tokens[j].type == ')') cnt--;
        if (cnt == 0) {
            i = j;
            break;
        }
    }
}
```

这样括号内部的运算符就不会错误地参与当前层主运算符的选择。

另外，本次实现里还有一个很重要的细节：单目运算符不是在一开始就处理，而是在“找不到二元主运算符”时才处理：

```
if (op == -1) {
    if (tokens[p].type == TK_NEG) { ... }
    if (tokens[p].type == TK_NOT) { ... }
    if (tokens[p].type == TK_BNOT) { ... }
    if (tokens[p].type == TK_DEREF) { ... }
}
```

这样写的好处是，像 `-1 + 2` 这样的表达式会先被 `+` 拆开，而不会被误解释成对整个 `1 + 2` 取负。这一点和上面主运算符优先于单目运算符的处理顺序是完全对应的。

本次实现里还单独对 `&&` 和 `||` 做了短路求值处理：

```
if (tokens[op].type == TK_AND) {
    uint32_t val1 = eval(p, op - 1, success);
    if (val1 == 0) return 0;
    uint32_t val2 = eval(op + 1, q, success);
    return (val2 != 0) ? 1 : 0;
}
if (tokens[op].type == TK_OR) {
    uint32_t val1 = eval(p, op - 1, success);
    if (val1 != 0) return 1;
    uint32_t val2 = eval(op + 1, q, success);
    return (val2 != 0) ? 1 : 0;
}
```

这说明当前表达式求值并不只是“把两边都算出来再算结果”，而是已经尽量向真实运算语义靠拢。虽然 PA1 的表达式子集远没有完整 C 语言那么复杂，但在逻辑运算这里已经体现出了比较标准的短路行为。

从指导书要求来看，表达式递归求值这一部分真正要解决的问题，不是“把某几个运算符算出来”，而是把表达式按照正确结构拆开。也就是说，递归求值的核心不是递归本身，而是“如何根据表达式结构决定递归边界”。这也是这部分实现最容易出错、同时也是最值得分析的地方。

6.3 单目负号、减号、乘法和指针解引用

`-` 与 `*` 的二义性如果处理不当，求值结果会很容易出错。

例如：

- `1 - 2` 中的 `-` 是二元减法。
- `-1` 中的 `-` 是单目负号。
- `2 * 3` 中的 `*` 是乘法。
- `*0x10000` 中的 `*` 是指针解引用。

这一部分在代码里其实分成两个层次处理：第一层是在 `make_token()` 中决定当前字符应该被识别成哪一种 token；第二层是在 `eval()` 中根据 token 类型决定如何计算。也就是说，`-` 和 `*` 的二义性不是在递归求值时临时猜测出来的，而是在词法分析阶段就被改写成不同的 token 类型。

词法分析中的关键代码如下：

```
case '-':
    if (nr_token == 0 || !is_value_token(tokens[nr_token - 1].type)) {
        if (!append_token(TK_NEG, "-", 1)) {
            return false;
        }
        break;
    }
case '*':
    if (nr_token == 0 || !is_value_token(tokens[nr_token - 1].type)) {
        if (!append_token(TK_DEREF, "*", 1)) {
            return false;
        }
        break;
    }
case '+':
case '&':
case '|':
case '^':
case '/':
case '%':
case '(':
case ')':
    if (!append_char_token(rules[i].token_type)) {
        return false;
    }
    break;
```

这段代码的判断依据是：如果 `-` 或 `*` 出现在表达式开头，或者它前面的 token 不是一个“值”，那么它就不可能是普通二元运算符，而应该被解释为单目运算符。这里“值”由 `is_value_token()` 判定，当前实现中包括：

```
static bool is_value_token(int type) {
    return type == TK_NUM || type == TK_REG || type == ')';
}
```


因此：

- 在 $1 - 2$ 中， $-$ 前面是数字 1 ，属于值，所以识别成普通减号；
- 在 -1 中， $-$ 位于表达式开头，所以识别成 `TK_NEG`；
- 在 $2 * 3$ 中， $*$ 前面是数字 2 ，属于值，所以识别成普通乘号；
- 在 $*0x100000$ 中， $*$ 位于表达式开头，所以识别成 `TK_DEREF`。

这种设计的好处是，递归求值阶段就不需要再根据原始字符去猜当前到底是减法、负号、乘法还是解引用，而是直接根据 token 类型处理。

在求值阶段，对应逻辑集中在 `eval()` 的 `op == -1` 分支中：

```
if (op == -1) {
    if (tokens[p].type == TK_NEG) {
        uint32_t val = eval(p + 1, q, success);
        if (!*success) { return 0; }
        return (uint32_t)(-val);
    }
    if (tokens[p].type == TK_NOT) {
        uint32_t val = eval(p + 1, q, success);
        if (!*success) { return 0; }
        return (val == 0) ? 1 : 0;
    }
    if (tokens[p].type == TK_BNOT) {
        uint32_t val = eval(p + 1, q, success);
        if (!*success) { return 0; }
        return ~val;
    }
    if (tokens[p].type == TK_DEREF) {
        uint32_t addr = eval(p + 1, q, success);
        if (!*success) { return 0; }
        return vaddr_read(addr, 4);
    }
}
```

这里之所以要等到 `op == -1` 时才处理这些单目运算符，是因为只要当前区间还能找到二元主运算符，就说明这一段表达式的主体结构还没有拆完，不能过早把开头的 $-$ 或 $*$ 当成整段表达式的根节点。这样做正是为了避免把：

$-1 + 2$

误解为：

$-(1 + 2)$

如果 $-1 + 2$ 一开始就把开头的 $-$ 当成整段表达式的根节点，就会先递归求 $1 + 2$ ，再整体取负，结果显然错误。当前实现先找二元主运算符，发现 $+$ 才是当前层主运算符，于是会先拆成：

$(-1) + 2$

左半部分再继续递归处理 TK_NEG ，这样运算顺序才正确。

对于 TK_DEREF 也是同样的思路。它并不是直接读取 $tokens[p + 1]$ 的值，而是先递归计算后面的整个子表达式：

```
uint32_t addr = eval(p + 1, q, success);  
return vaddr_read(addr, 4);
```

这意味着像：

$*(\$eax + 4)$

这样的表达式也能成立，因为 $eval(p + 1, q, success)$ 先把 $(\$eax + 4)$ 算成一个地址，然后再用这个地址去读 4 字节内存。

这里也可以更清楚地看出指导书中的取舍：PA1 中的表达式求值不是在完整模拟 C 语言表达式，而是在实现一个“足够支撑调试器使用”的表达式子集。因此所有计算结果统一当作 $uint32_t$ 处理，解引用也统一看成读取一个 4 字节整数，而没有引入更完整的类型系统。这种取舍是符合 PA1 实验目标的。

6.4 p 表达式求值命令

为了让表达式求值模块能够直接在 monitor 中测试，还新增了：

p EXPR

对应实现的核心逻辑如下：

```
bool success = false;
uint32_t value = expr(args, &success);
if (!success) {
    printf("Bad expression: %s\n", args);
    return 0;
}

printf("0x%08x\t%u\n", value, value);
```

也就是说，**p** 命令本身并不参与求值，它只是把用户输入交给 **expr()**，再把结果以十六进制和十进制形式打印出来。

这个命令虽然简单，但它非常有用。因为如果没有 **p**，很多表达式逻辑只能靠 **x** 或监视点间接测试，调试过程会明显绕远。而有了 **p**，就可以把表达式系统单独测通，再让其他命令建立在它之上。

实现 **p** 命令之后，前面讨论过的一些特殊情况就可以被直接拿来验证，而不必再借助内存扫描或监视点间接观察。例如，可以直接测试十六进制常量、寄存器参与运算、括号优先级、单目负号、连续负号、逻辑运算以及指针解引用等表达式，从而检查词法分析和递归求值是否一致地工作。这样一来，表达式系统中的很多边界情况都能够被单独拿出来做针对性验证。

下图展示的是 **p** 命令在正常表达式上的测试结果。通过这些测试可以验证前面实现的加减乘除、括号优先级、寄存器取值和十六进制数字识别是否正确，也能直观看到表达式最终被计算成了怎样的十六进制和十进制结果。

```
[src/monitor/monitor.c,32,welcome] Build time: 05:46:12, Apr 22 2026
For help, type "help"
(nemu) p (1+2)*3
[src/monitor/debug/expr.c,157,make_token] match rules[16] = "(" at position 0 with len 1: (
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 1 with len 1: 1
[src/monitor/debug/expr.c,157,make_token] match rules[7] = "+" at position 2 with len 1: +
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 3 with len 1: 2
[src/monitor/debug/expr.c,157,make_token] match rules[17] = ")" at position 4 with len 1: )
[src/monitor/debug/expr.c,157,make_token] match rules[9] = "*" at position 5 with len 1: *
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 6 with len 1: 3
0x00000009      9
(nemu) p -1+2
[src/monitor/debug/expr.c,157,make_token] match rules[8] = "-" at position 0 with len 1: -
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 1 with len 1: 1
[src/monitor/debug/expr.c,157,make_token] match rules[7] = "+" at position 2 with len 1: +
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 3 with len 1: 2
0x00000001      1
(nemu) p *0x100000
[src/monitor/debug/expr.c,157,make_token] match rules[9] = "*" at position 0 with len 1: *
[src/monitor/debug/expr.c,157,make_token] match rules[19] = "0[xX][0-9a-fA-F]+" at position 1 with len 8: 0x100000
0x001234b8     1193144
(nemu) p %eax+1
[src/monitor/debug/expr.c,157,make_token] match rules[18] = "%(eip|eax|ecx|edx|ebx|esp|ebp|esi|edi)" at position 0 with len 4: %eax
[src/monitor/debug/expr.c,157,make_token] match rules[7] = "+" at position 4 with len 1: +
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 5 with len 1: 1
0x4324b894     112648020
(nemu) p --1
[src/monitor/debug/expr.c,157,make_token] match rules[8] = "-" at position 0 with len 1: -
[src/monitor/debug/expr.c,157,make_token] match rules[8] = "-" at position 1 with len 1: -
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 2 with len 1: 1
0x00000001      1
(nemu) █
```

下图展示的是 **p** 命令在非法或特殊表达式上的测试结果。这里更适合用来验证前面专门处理过的边界情况，例如括号不匹配、表达式缺少操作数、单目负号与二元减号的区分

是否正确，以及错误输入时 `expr()` 是否能够返回失败而不是让 monitor 崩溃。也就是说，这张图对应的不是“普通表达式能算出来”，而是“特殊情况和错误情况也能被正确处理”。

```
[src/monitor/monitor.c,50,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 05:46:12, Apr 22 2026
For help, type "help"
(nemu) p 1+2
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,157,make_token] match rules[7] = "+" at position 1 with len 1: +
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 2 with len 1: 2
0x00000003      3
(nemu) p 1 +
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,157,make_token] match rules[0] = " " at position 1 with len 1:
[src/monitor/debug/expr.c,157,make_token] match rules[7] = "+" at position 2 with len 1: +
Error: missing operand
Bad expression: 1 +
(nemu) p *
[src/monitor/debug/expr.c,157,make_token] match rules[9] = "\"*" at position 0 with len 1: *
Error: invalid single token '*'
Bad expression: *
(nemu) p 1 + )
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 0 with len 1: 1
[src/monitor/debug/expr.c,157,make_token] match rules[0] = " " at position 1 with len 1:
[src/monitor/debug/expr.c,157,make_token] match rules[7] = "+" at position 2 with len 1: +
[src/monitor/debug/expr.c,157,make_token] match rules[0] = " " at position 3 with len 1:
[src/monitor/debug/expr.c,157,make_token] match rules[17] = ")" at position 4 with len 1: )
Error: parentheses not matched
Bad expression: 1 + )
(nemu) p 4/0
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 0 with len 1: 4
[src/monitor/debug/expr.c,157,make_token] match rules[10] = "/" at position 1 with len 1: /
[src/monitor/debug/expr.c,157,make_token] match rules[20] = "[0-9]+" at position 2 with len 1: 0
Error: division by zero
Bad expression: 4/0
```

6.5 表达式求值与编译器前端的联系

通过实现表达式求值，可以更直观地看到编译器前端的一部分工作方式。表达式求值的过程本质上分成两步：

- 词法分析：把字符串切成 token。
- 结构分析与求值：根据 token 之间的关系确定计算方式。

这与编译器处理源代码的方式非常相似。编译器也要先识别关键字、标识符、数字常量、运算符等 token，再进一步分析程序结构。不同之处只在于，PA1 处理的是一个极小的表达式子集，而编译器要处理完整语言。

为了更直观地理解这一点，这里额外实现了一个简单的语法高亮小演示。这个演示的目标并不是生成完整编辑器，而是验证“词法分析先于后续处理”这一思路：先把代码字符串切分成不同类型的 token，再根据 token 类型选择不同颜色输出。换句话说，这个演示并不直接参与 NEMU 的表达式求值，但它很好地展示了词法分析在另一个场景中的应用方式。

语法高亮脚本中的核心规则如下：

```

RULES = [
    ("comment", r"//[^\n]*|/\*.*?\*/"),
    ("string", r'"([^"\\]|\\.)*"'),
    ("keyword", r"\b(if|else|for|while|return|break|continue|sizeof)\b"),
    ("type", r"\b(int|char|void|uint32_t|bool|static|const|struct)\b"),
    ("number", r"\b0[xX][0-9a-fA-F]+\b|\b[0-9]+\b"),
    ("operator", r"==|!=|<=|>=|&&|\||\&|\<|\>|[\+\-\*\/%<>=!&^~]"),
    ("space", r"\s+"),
    ("identifier", r"[A-Za-z_][A-Za-z0-9_]*"),
    ("punct", r"[{}()\\\.,;:]"),
]

```

主处理流程则是：

```

def highlight(code):
    pos = 0
    while pos < len(code):
        for kind, pattern in RULES:
            match = re.match(pattern, code[pos:], re.S)
            if match is None:
                continue

            text = match.group(0)
            if kind in COLORS:
                sys.stdout.write(COLORS[kind] + text + COLORS["reset"])
            else:
                sys.stdout.write(text)
            pos += len(text)
            break
        else:
            sys.stdout.write(code[pos])
            pos += 1

```

这段代码和 `expr.c` 中 `make_token()` 的工作方式非常接近：都是维护一个当前位置 `pos`，然后用一组规则从当前位置开始匹配；一旦匹配成功，就把这一段文本作为一个 token 处理，再把位置移动到下一个待识别区域。差别只在于：

- `make_token()` 在识别后把 token 记录进 `tokens[]`，供后续 `eval()` 计算。
- `highlight()` 在识别后直接按 token 类型输出不同颜色，供人观察。

从实现角度看，这个高亮程序最关键的地方有三点：

1. 先写出一组足够覆盖常见 C 代码成分的规则，例如注释、字符串、关键字、类型、数字、运算符、标识符和标点。
2. 再从当前位置开始按顺序尝试这些规则，谁先匹配成功，就把这一段文本识别成对应 token。

3. 最后根据 token 类型输出不同颜色，而不是继续做语法分析或求值。

规则顺序在这里尤其重要。例如：

- `==`、`!=`、`<=`、`>=` 这一类多字符运算符必须写在前面，否则会被拆成两个单字符运算符；
- 十六进制数字规则必须早于普通十进制数字规则，否则 `0x1000` 可能会先被错误识别成 `0`；
- 关键字规则和标识符规则也要区分清楚，否则 `return`、`while` 这一类词就会被普通变量名规则吞掉。

为了验证这个思路，可以把一段比较简单但成分较全的 C 代码作为输入，例如其中同时包含：

- `int`、`return`、`static` 这一类词会被识别成关键字或类型；
- `foo`、`main`、`a`、`b`、`x`、`y` 会被识别成标识符；
- `3`、`1`、`2` 会被识别成数字；
- `+`、`*`、`=` 会被识别成运算符；
- `"b = %d\\n"` 会被识别成字符串。

从这个结果可以看出，语法高亮和表达式求值虽然用途不同，但最底层都依赖于“先把输入字符串按规则切成 token，再决定后续如何处理”。表达式求值是在 token 基础上继续做结构分析和递归计算，而语法高亮是在 token 基础上继续做分类着色。两者的后续目标不同，但它们共享了编译器前端最基础的第一步，也就是词法分析。

```
static uint32_t eval(int p, int q) {
    // choose the dominant operator first
    if (p == q) {
        return strtoul(tokens[p].str, NULL, 0);
    }

    if (tokens[p].type == TK_DEREF) {
        uint32_t addr = eval(p + 1, q);
        return vaddr_read(addr, 4);
    }

    return 1 + 2 * 3 <= 7 && 10 % 3 != 0;
}
```

6.6 阶段二小结

这一阶段的关键代码可以概括为三部分。第一，表达式系统必须拆成“词法分析”和“递归求值”两步，如果没有词法分析，后续求值就只能直接在原始字符串上做字符级判断，逻辑会非常混乱；第二，`-` 和 `*` 的二义性必须在词法分析阶段先处理，否则 `-1 + 2`、`*0x100000` 这类表达式很容易被错误理解；第三，`p EXPR` 命令把表达式模块独立暴露出来，使其可以在监视点接入之前先单独测试。

这一阶段的典型问题主要是边界条件。比如 token 缓冲区不能只靠扩大数组来降低出错概率，而应在每次写入前显式检查；单目负号不能在表达式开头就粗暴地对整段后缀取负，否则会把 `-1 + 2` 误算成 `-(1 + 2)`；十六进制数字规则和十进制规则的顺序也不能放错，否则 `0x100000` 这种最常见的地址写法都无法正确识别。

从测试结果来看，阶段二的实现已经具备较好的稳定性。基础四则运算、括号表达式、十六进制常量、寄存器参与计算、单目负号、解引用表达式都可以在 `p EXPR` 中得到正确结果；非法表达式则会返回错误并重新回到 monitor，而不会导致程序崩溃。这说明词法分析、主运算符选择、递归求值和错误处理已经基本形成闭环。

阶段二完成后，monitor 已经不仅能执行和查看程序状态，还能理解用户输入的表达式，并基于表达式来支持内存扫描、表达式打印和后续监视点功能。相比阶段一，这一阶段更偏“字符串处理和结构分析”，但它同样是调试器的关键能力，因为没有表达式系统，很多调试命令都会失去灵活性。

7. 阶段三：监视点

7.1 监视点池管理

监视点功能的核心在于：预先准备一组可复用的节点，再通过链表在“空闲”和“使用中”两种状态之间切换，而不是每次创建监视点时临时申请内存。当前实现一开始就定义了固定大小的监视点池：

```
#define NR_WP 32
static WP wp_pool[NR_WP];
static WP *head, *free_;
```

这里的 `static` 不是随意加上的。它的作用是把 `wp_pool`、`head` 和 `free_` 限制在 `watchpoint.c` 内部，避免其他文件直接改动监视点链表结构。这样一来，外部模块只能通过 `new_wp()`、`free_wp()`、`insert_wp()`、`delete_wp()` 这些接口操作监视点，状态管理会更稳定。

初始化时，全部节点都被挂到空闲链表 `free_` 上；真正创建监视点时，再把节点从空闲链表摘下并挂到使用链表 `head` 上。这个过程在代码里体现得很直接：

```
WP* new_wp() {
    if (free_ == NULL) {
        printf("No more watchpoint!\n");
        return NULL;
    }

    WP* wp = free_;
    free_ = free_>next;
    wp->next = head;
    head = wp;
    return wp;
}
```

这段实现说明，监视点池管理的重点并不在复杂的数据结构技巧，而在于能否把节点状态切换写得简单、稳定、可控。PA1 这里采用固定节点池而不是动态分配，正好符合指导书的训练目的：先把基础机制做对，而不是过早引入额外复杂性。

删除监视点时则执行相反操作：先把节点从使用链表中摘掉，再重新塞回空闲链表头部。这样一来，同一批节点就能被反复复用，而不会随着创建和删除不断申请和释放内存。

这一部分还有一个实现细节容易被忽略：监视点池不是在第一次使用时才初始化，而是在 NEMU 启动阶段就已经完成初始化。也就是说，monitor 一进入工作状态，监视点池已经处于“可立即分配”的状态。这样做的好处是，后续 `w` 命令只需要专注于“申请节点并绑定表达式”，而不必再混杂额外的初始化逻辑。对于调试器这种基础设施来说，初始化时机越明确，后面越不容易出隐蔽问题。

另外，当前实现中一个很重要的细节是：监视点保存的不是“某个寄存器编号”或“某个固定地址”，而是整段表达式字符串。对应代码如下：

```
strncpy(wp->expr, args, sizeof(wp->expr) - 1);
wp->expr[sizeof(wp->expr) - 1] = '\0';
```

这样写的意义在于，监视点可以直接支持：

```
w $eax + 1
```


这类带空格的表达式。如果这里只保存第一个单词，那么监视点功能就会被迫退化成“只能观察一个寄存器或一个常量地址”，显然达不到指导书希望实现的调试效果。

从命令入口看，监视点相关功能也是通过 monitor 接入的。在 `ui.c` 中，`w`、`d`、`info` 这几个接口本身并不负责维护链表，它们只负责解析用户输入并调用监视点模块提供的函数。例如：

```
static int cmd_w(char *args) {
    if (args == NULL) {
        printf("Usage: w EXPR\n");
    }
    else {
        insert_wp(args);
    }
    return 0;
}

static int cmd_d(char *args) {
    if (args == NULL) {
        printf("Usage: d N\n");
    }
    else {
        int no = atoi(args);
        delete_wp(no);
    }
    return 0;
}
```

这种分层方式比较重要：`ui.c` 只关心“用户输入了什么命令”，而 `watchpoint.c` 才真正关心“监视点链表如何维护”。这样做的结果是，命令解析和监视点管理两个层次的责任是分开的，后续修改其中一部分时，不容易影响另一部分。

7.2 监视点触发机制

监视点命令主要包括：

- `w EXPR`：新建监视点。
- `info w`：打印全部监视点。
- `d N`：删除编号为 `N` 的监视点。

监视点真正发挥作用的关键并不在“创建成功”，而在“程序运行过程中能否持续检查并及时停下”。当前实现中，插入监视点时会先计算一次表达式初值并保存下来：

```

bool success = true;
wp->value = expr(wp->expr, &success);
if (!success) {
    printf("Invalid expression!\\n");
    free_wp(wp);
    return;
}

```

这样做的意义在于，监视点从创建那一刻开始就已经绑定了“旧值”。后续每执行完一条客户指令，只要重新计算一次表达式并与旧值比较，就可以知道程序状态是否发生了变化。

核心逻辑如下：

```

uint32_t value = expr(temp->expr, &success);

if (value != temp->value) {
    printf("Watchpoint %d triggered: %s\\n", temp->NO, temp->expr);
    printf("old value = 0x%08x, new value = 0x%08x\\n", temp->value, value);
    temp->value = value;
    triggered = true;
}

```

这段代码说明，监视点机制本质上是“表达式复算 + 新旧值比较”。它依赖的不是某条特殊指令，而是前一阶段已经实现好的表达式求值系统。也正因为如此，指导书才会先安排表达式求值，再安排监视点；如果没有 `expr()`，监视点就无法支持任意表达式。

一旦至少有一个监视点被触发，`cpu_exec()` 就会把 NEMU 状态切回停止，返回 monitor。这样程序就不再需要盲目地一条条单步跟踪，而是可以在“目标表达式第一次发生变化”的位置直接停下来。

这里最关键的一步，是把监视点检查真正接到执行循环里，而不是只在某个命令里手动调用。当前实现是在 `cpu_exec()` 每执行完一条指令之后，立即重新检查全部监视点：

```
for (; n > 0; n--) {
    exec_wrapper(print_flag);

    if (check_watchpoints()) {
        nemu_state = NEMU_STOP;
        printf("Watchpoints were triggered.\n");
    }

    if (nemu_state != NEMU_RUNNING) { return; }
}
```

这段接线非常关键，因为它决定了监视点到底是“真正自动触发”，还是“只能被动查询”。只有把 `check_watchpoints()` 放进每条指令执行后的路径里，监视点才真正具备调试器意义上的自动停机能力。否则，即使前面监视点池、表达式求值和比较逻辑都写对了，也只能算一个“可以手动检查的表达式记录器”，而不是完整的监视点功能。

从完整实现链路来看，阶段三实际上包含了三步：

1. 在 `monitor` 初始化时准备好监视点池。
2. 在 `ui.c` 中通过 `w`、`d`、`info w` 提供用户接口。
3. 在 `cpu_exec()` 中把监视点检查嵌入每条指令执行后的路径。

也正因为这三步都接好了，监视点才不是孤立的一个数据结构，而是变成了整个调试流程中的实际可用功能。

值得注意的是，如果同一条指令导致多个监视点同时变化，这里会选择全部打印出来，而不是遇到第一个就提前返回。这样做的好处是信息更完整，也更符合调试场景中的真实需要。

指导书在这一阶段提出监视点功能，本质上是在强调调试器的一个重要思想：不是所有调试都靠人工不停单步，而是可以把“程序状态何时变化”这个判断条件也交给调试器。

```

[src/monitor/monitor.c,50,load_default_img] No image is given. Use the default build-in image.
Welcome to NEMU!
[src/monitor/monitor.c,32,welcome] Build time: 05:46:12, Apr 22 2026
For help, type "help"
(nemu) info w
No watchpoint!
(nemu) w $eax
[src/monitor/debug/expr.c,161,make_token] match rules[23] = "\\$(eip|eax|ecx|edx|ebx|esp|ebp|esi|edi)"
at position 0 with len 4: $eax
Watchpoint 0: $eax = 0x43cc39b5
(nemu) info w
NO      VALUE      EXPR
0       0x43cc39b5  $eax
(nemu) si
100000:  b8 34 12 00 00          movl $0x1234,%eax
[src/monitor/debug/expr.c,161,make_token] match rules[23] = "\\$(eip|eax|ecx|edx|ebx|esp|ebp|esi|edi)"
at position 0 with len 4: $eax
Watchpoint 0 triggered: $eax
old value = 0x43cc39b5, new value = 0x00001234
Watchpoints were triggered.
(nemu) info w
NO      VALUE      EXPR
0       0x00001234  $eax
(nemu) d 0
Watchpoint 0 deleted
(nemu) info w
No watchpoint!
(nemu)

```

7.3 软件断点与 `int3` 的一点理解

在整理监视点和调试机制时，也进一步补充了对软件断点基本原理的理解。x86 中 `int3` 指令的操作码是 `0xcc`，长度只有 1 字节。这一点非常关键，因为调试器设置软件断点时，通常只需要把目标地址的第一个字节替换成 `0xcc`，程序执行到该位置时就会触发调试异常。后续调试器再恢复原字节，并调整执行位置，就可以继续正常运行。

如果 `int3` 不是 1 字节，而是更长，那么在任意指令首字节处插入软件断点就会复杂得多，因为必须覆盖更多原始机器码字节，恢复和继续执行的难度也会明显增加。

还可以用 GDB 做一个实验：如果故意把断点下在一条指令的中间，而不是首字节位置，那么程序很可能不会正常停在断点上，而是因为指令编码被破坏，最终触发异常。这说明软件断点设置必须严格遵守指令边界。

```

(gdb) break *main+5
Breakpoint 1 at 0x8048426: file some_program.c, line 8.
(gdb) x/16xb main
0x8048421 <main>:      0x8d    0x4c    0x24    0x04    0x83    0xe4    0xf0    0xff
0x8048429 <main+8>:   0x71    0xfc    0x55    0x89    0xe5    0x51    0x83    0xec
(gdb) run
Starting program: /home/linshangjin/code/PA报告/tools/gdb-break-demo/some_program

Program received signal SIGSEGV, Segmentation fault.
0x08048428 in main () at some_program.c:8
8      int main() {

```

7.4 阶段三小结

阶段三的实现把简易调试器从“能看状态”进一步推进到了“能根据状态变化自动停下”。真正起作用的关键代码并不多，但它把前面已经完成的几项能力串了起来：固定大小的监

视点池负责稳定管理节点，表达式求值负责把用户输入的表达式转成可比较的值，而 `cpu_exec()` 则负责在每条指令之后触发检查。

这一阶段最容易出错的地方，恰好也反映了监视点机制的核心：如果创建监视点时没有保存完整表达式，那么监视对象就会被错误截断；如果检查逻辑不是每条指令后都执行，那么监视点就失去了“及时停下”的意义；如果发现第一个变化就立即返回，又可能漏掉同一条指令造成的其他变化。也就是说，监视点虽然从接口上看只是 `w`、`info w`、`d` 三个命令，但真正保证它可靠工作的，是后面那套链表管理、表达式复算和逐条检查的配合关系。

从实验结果来看，当前实现已经能够完成完整的监视点 workflow：设置监视点、执行若干条指令、观察表达式值变化、删除监视点并重新检查。表达式值变化时，NEMU 能及时停止并打印旧值和新值，这说明监视点池管理、表达式复算与触发检查逻辑已经连通。再结合对 `int3` 和软件断点机制的补充理解，可以更清楚地看到：调试器真正有价值的地方，并不只是“显示信息”，而是能够在合适的时刻把程序执行精确地停在需要观察的位置上。

8. 必答题

8.1 i386 手册查阅位置

这一部分查阅时使用的是《Intel 80386 Programmer's Reference Manual》。实际查找时，先根据问题类型在目录中定位大章节，再进入对应小节继续缩小范围。这样查起来比直接全文搜索更清晰，也更容易把“需要阅读的范围”写准确。

从目录可以看到，这三个问题分别落在三类内容里：

- 标志位含义：Chapter 2 Basic Programming Model
- 指令编码格式：Chapter 2 Instruction Format 与 Chapter 17 Instruction Set
- 具体指令格式：Chapter 17 80386 Instruction Set 中的 MOV 条目

按这个思路，三个问题可以定位如下。

EFLAGS 中的 CF 位是什么意思

这个问题属于“寄存器中某一位的含义”，因此不应该从某条具体指令反推，而应该先查寄存器与程序运行模型相关章节。目录页给出的入口是：

Chapter 2 BASIC PROGRAMMING MODEL	
2.3.4 Flags Register	page 33
2.3.4.1 Status Flags	page 34

因此，阅读范围可写为：

Intel 80386 Programmer's Reference Manual	
Chapter 2 Basic Programming Model	
Section 2.3.4 Flags Register (手册第 33 页)	
Section 2.3.4.1 Status Flags (手册第 34 页)	

这两页已经足够回答问题：

- 第 33 页先说明 **EFLAGS** 是 32 位寄存器，并给出它的整体作用；
- 第 34 页的 **Figure 2-8** 直接标出 **CF** 是 **Carry Flag**；
- 同页的 **2.3.4.1 Status Flags** 说明它属于状态标志位，用于让一条指令的执行结果影响后续指令。

结合手册内容，可以把答案写成：**CF** 即 **Carry Flag**，表示无符号运算中的进位或借位。在加法中，如果最高位产生进位，则 **CF=1**；在减法中，如果发生借位，则 **CF=1**。后续很多条件转移和算术相关指令都会依赖这个标志位。

ModR/M 字节是什么

这个问题属于“指令编码格式”问题。查阅时如果直接从某条指令入手，容易只看到局部编码，而看不到 **ModR/M** 在整个 x86 编码体系中的作用。因此应先从手册目录中的格式章节进入。

目录页给出的入口有两个：

Chapter 2 Basic Programming Model	
2.4 Instruction Format	page 35
2.5.3 Memory Operands	page 38
Chapter 17 80386 Instruction Set	
17.2 Instruction Format	page 240
17.2.2.1 Opcode	page 246

因此，比较完整的阅读范围可以写为：

Intel 80386 Programmer's Reference Manual
Chapter 2
Section 2.4 Instruction Format (手册第 35–36 页)
Section 2.5.3 Memory Operands (手册第 38 页)

Chapter 17
Section 17.2 Instruction Format (手册第 240–245 页)
Section 17.2.2.1 Opcode (手册第 246 页)

其中最关键的内容是：

- 第 35 至 36 页先从整体上说明一条指令由 opcode、寄存器说明、寻址方式说明、SIB、位移量、立即数组成；
- 第 38 页明确指出：访问内存的很多数据操作指令会在 opcode 后跟一个 **modR/M byte**；
- 第 240 至 245 页正式给出 **Instruction Format**、**Figure 17–1**、**Figure 17–2** 以及 **ModR/M / SIB** 表；
- 第 246 页解释 **/digit** 和 **/r** 这样的记号，说明它们如何依赖 **ModR/M** 的不同字段。

据此可以把 **ModR/M** 概括为：它是 x86 指令编码中的一个说明字节，通常分成 **mod**、**reg/opcode**、**r/m** 三部分，用来描述操作数是寄存器还是内存、寄存器编号是什么、内存寻址方式是什么。在 NEMU 后续实现指令译码时，很多操作都必须先解析 **ModR/M**，才能真正知道指令要访问哪个寄存器或哪个内存地址。

mov 指令的具体格式

这个问题属于“具体指令条目”的查阅，最合理的方法是先从目录中找到 **MOV** 在指令集中的页码，再回到该条目查看所有变体，而不是只记住一种最常见形式。

在手册目录中，**MOV** 位于：

Chapter 17 80386 Instruction Set
MOV -- Move Data page 345

因此，阅读范围可写为：

Intel 80386 Programmer's Reference Manual
Chapter 17 80386 Instruction Set
MOV -- Move Data (手册第 345 页，必要时连同其后说明一起阅读)

第 345 页的 **MOV** 条目直接给出了多种编码形式，例如：

```
MOV r/m8, r8
MOV r/m16, r16
MOV r/m32, r32
MOV r8, r/m8
MOV r16, r/m16
MOV r32, r/m32
MOV AL, moffs8
MOV AX, moffs16
MOV EAX, moffs32
MOV moffs8, AL
MOV moffs16, AX
MOV moffs32, EAX
MOV reg8, imm8
MOV reg16, imm16
MOV reg32, imm32
MOV r/m8, imm8
MOV r/m16, imm16
MOV r/m32, imm32
```

从这一页可以直接看出三点：

- **MOV** 不是单一格式，而是一组“数据传送”指令的总称；
- 有些形式依赖 **ModR/M**，例如 **MOV r/m32, r32**、**MOV r32, r/m32**；
- 也有一些形式不使用 **ModR/M**，例如 **MOV EAX, moffs32** 和 **MOV reg32, imm32**。

因此，如果报告里要回答“**mov** 指令的具体格式是怎么样”的，更准确的写法不是只写一条形式，而是说明 **MOV** 包含寄存器到寄存器、寄存器到内存、内存到寄存器、立即数到寄存器、立即数到内存、**moffs** 特殊寻址等多种编码格式，并指出这些格式在手册第 345 页的表格中已经逐项列出。

8.2 代码行数统计

讲义这一问实际上包含四个小问题：

1. **nemu/** 目录下所有 **.c** 和 **.h** 文件总共有多少行代码？
2. 使用了什么 shell 命令得到这个结果？
3. 和框架代码相比，PA1 中编写了多少行代码？
4. 除去空行之后，代码总行数是多少？

这部分不只是让报告给出数字，还要求把“如何统计”的命令写出来。因此，下面分别给出实际使用的命令和结果。

1. 统计 .c 和 .h 文件总行数

使用的命令如下：

```
find nemu \( -name '*.c' -o -name '*.h' \) -print0 | xargs -0 wc -l
```

其中：

- `find nemu` 用于递归找到 `nemu/` 目录下所有 `.c` 和 `.h` 文件；
- `-print0` 与 `xargs -0` 配合，避免文件名中如果出现空格时被错误切分；
- `wc -l` 用于统计每个文件的行数；
- 最后一行的 `total` 就是总行数。

实际统计结果为：

```
4272 total
```

也就是说，完成 PA1 内容之后，`nemu/` 目录下所有 `.c` 和 `.h` 文件总共有 4272 行。

2. 统计去除空行后的总行数

使用的命令如下：

```
find nemu \( -name '*.c' -o -name '*.h' \) -print0 | xargs -0 sed  
'/^[[[:space:]]*$/d' | wc -l
```

这里的核心做法是：

- `sed '/^[[[:space:]]*$/d'` 删除所有空行；
- 再交给 `wc -l` 统计剩余行数。

实际统计结果为：

```
3539
```

也就是说，去掉空行之后，`nemu/` 目录下所有 `.c` 和 `.h` 文件共有 3539 行。

3. 与框架代码相比，PA1 中新增了多少行代码

讲义提示“2017 分支中记录的正好是做 PA1 之前的状态”，这里的关键在于如何“回到过去”。在当前工程中，通过查看 Git 分支和提交记录，可以定位到：

```
48accd8 (origin/pa0, pa0) before starting pa1
```

也就是说，**pa0** 分支上的这次提交正对应 PA1 开始前的基线状态。于是，与框架代码相比的改动量可以通过下面的命令统计：

```
git diff --shortstat 48accd8 -- nemu
```

实际结果为：

```
9 files changed, 833 insertions(+), 42 deletions(-)
```

这说明相对于 PA1 开始前的框架代码，当前 **nemu/** 目录在 PA1 中共新增 **833** 行，删除 **42** 行。

如果进一步按文件查看各部分改动量，可以使用：

```
git diff --numstat 48accd8 -- nemu
```

实际结果显示改动主要集中在：

```
nemu/src/monitor/debug/expr.c  
nemu/src/monitor/debug/ui.c  
nemu/src/monitor/debug/watchpoint.c  
nemu/include/cpu/reg.h
```

这也与 PA1 的实验内容完全一致，即寄存器结构、表达式求值、调试器命令和监视点是主要实现区域。

4. 把统计命令写入 Makefile

为了后续实验能继续方便统计代码行数，可以把统计命令写进 `nemu/Makefile`。当前 `Makefile` 中已经加入：

```
count:
    @echo "All .c/.h lines:"
    @find . \( -name "*.c" -o -name "*.h" \) -print0 | xargs -0 wc -l |
tail -1
    @echo "Non-empty .c/.h lines:"
    @find . \( -name "*.c" -o -name "*.h" \) -print0 | xargs -0 sed
'/^[[:space:]]*$$/d' | wc -l
```

这样后续只要在 `nemu/` 目录下执行：

```
make count
```

就可以同时得到总行数和非空行数，不需要每次再手动重写长命令。

8.3 `-Wall` 与 `-Werror`

`Makefile` 中的编译选项包含：

```
CFLAGS += -O2 -MMD -Wall -Werror -ggdb $(INCLUDES)
```

这两个选项之所以会出现在 `Makefile` 里，并不是为了“让编译看起来更严格”，而是因为 PA 的工程本身就是一个底层系统项目，很多 warning 往往就是 bug 的前兆。因此，这一问除了回答“它们是什么意思”，还应该回答“为什么实验工程要强制这样做”。

其中：

- `-Wall`：打开一组常见且有价值的编译警告。
- `-Werror`：把警告视为错误，只要出现 warning 就停止编译。

可以进一步具体说明如下。

`-Wall` 的作用

`-Wall` 并不是“打开 GCC 的所有警告”，而是打开一组最常见、最有价值的警告集合。它通常可以帮助尽早发现下面这些问题：

- 变量定义了但没有使用；
- 函数声明不完整或不一致；
- 不同类型之间的可疑转换；
- 格式化输出参数与格式串不匹配；
- 某些可能被忽略的逻辑错误。

对于 NEMU 这种系统代码来说，这些 warning 往往不是“无关紧要的小提示”，而很可能直接对应真实错误。

-Werror 的作用

-Werror 的作用是把 warning 提升为 error。也就是说，只要编译过程中出现 warning，编译就直接失败，而不是继续生成可执行文件。

它的意义在于：不允许带着“已知不干净的状态”继续往后做实验。否则，warning 很容易在一开始被忽略，等到后面行为异常时又很难回头定位。

为什么要同时使用 -Wall 和 -Werror

如果只用 -Wall，编译器虽然会提示问题，但这些提示很容易被忽略；如果只用 -Werror 而不开启足够多的 warning，那么很多潜在问题也根本不会被暴露出来。只有把两者结合起来，才能形成下面这套约束：

1. 先尽量多地发现可疑问题；
2. 再强制这些问题在编译阶段就被解决。

对于 PA 而言，这样做的价值非常直接：

- 可以尽早暴露低级错误，减少后续调试成本；
- 能迫使代码保持相对干净，避免“先凑合跑起来再说”的习惯；
- 在实现寄存器、表达式求值、监视点等功能时，如果有类型不匹配、格式输出错误或未使用变量等问题，会在最早阶段就被发现。

因此，-Wall 和 -Werror 一起使用，本质上是把“尽早发现问题、尽早修正问题”固化成构建规则的一部分。这种规则对后续所有 PA 阶段都有帮助，而不仅仅是 PA1。

9. 实验过程中遇到的问题与解决方法

这一部分单独整理了 PA1 中印象最深、调试时间也最长的几个问题。前面的阶段一、阶段二、阶段三已经分别说明了实现思路，这里更侧重记录“问题是怎么暴露出来的、当时

是怎么判断原因的、最后又是如何修掉的”。

9.1 CPU_state 组织不正确会导致 NEMU 一启动就失败

刚开始实现寄存器结构时，最大的误区是把 32 位、16 位、8 位寄存器当成三组彼此独立的数据去定义。这样写在表面上看起来“字段都全了”，但一旦运行到 `reg_test()`，NEMU 启动阶段就会直接触发断言失败，根本进不了 monitor。

回头看这个问题，核心原因是对 x86 寄存器之间的重叠关系理解得不够准确。`eax`、`ax`、`al`、`ah` 并不是四个独立寄存器，而是同一块底层存储的不同视图。如果用普通 `struct` 把它们拆开，那么修改 `eax` 时不会自动反映到 `ax`、`al`、`ah`，`reg_test()` 自然无法通过。

这个问题最后是通过重新整理 `reg.h` 中的数据布局解决的。把 32 位、16 位、8 位访问方式统一放进同一组 `union` 后，寄存器之间才能真正共享内存。重新编译运行后，`reg_test()` 能够通过，NEMU 才算真正完成了 PA1 的第一步。

这个问题也让我对 `union` 的作用有了更具体的理解。以前更多是把它当作一种“语法形式”来记，这次在寄存器建模里才真正体会到：`union` 的关键意义不是省空间，而是让同一段二进制数据能够被不同位宽、不同字段方式解释。

9.2 si 命令的参数处理一开始考虑得不完整

`si` 看起来是 PA1 里最简单的命令之一，但真正写的时候，参数边界条件并没有一开始就处理完整。最先遇到的问题是：`si` 既支持不带参数，也支持带数字参数；如果直接把参数交给整数转换函数，就会遇到空参数、非法参数甚至 `si 0` 这样的情况。

一开始如果只关注“把命令跑起来”，很容易写成“有参数就转数字，没有参数就凑合处理”的形式，但这种实现不够稳定。调试器命令本身就属于交互接口，如果参数处理做得含糊，后面排查问题时反而容易把错误归到执行框架上。

最后的处理方式是把情况分清楚：

- 没有参数时，默认执行 1 条指令；
- 参数是正整数时，执行对应条数；
- 参数不合法时，直接给出提示，不继续执行。

这样改完以后，`si` 的行为就比较稳定了。这个问题虽然不算复杂，但它提醒我：`monitor` 命令的实现不只是“调一个函数”，还要把用户输入的边界情况一起考虑进去。

9.3 `x` 命令最开始没有把表达式和地址真正连接起来

实现 `x` 命令时，最初的想法比较直接，就是读一个起始地址再连续打印内存。但实际对照指导书和 GDB 的行为之后，很快发现这里不能只把第二个参数当作“纯数字地址”，而应该把它当成表达式来求值。

如果这里偷懒，只支持十进制数字，那么很多调试场景都会非常别扭。例如 NEMU 里大量地址天然就是十六进制表示，寄存器值也常常要直接参与地址计算。如果 `x` 不能接受表达式，那么前面实现的表达式求值能力就无法真正服务于调试器。

因此后面把 `x` 的实现改成了两步：

- 先解析读取长度 `N`；
- 再把后面的参数交给 `expr()` 计算起始地址。

改完之后，`x 10 0x100000`、`x 4 $eax` 这样的形式都可以正常工作，`x` 命令才真正和 `monitor` 里的其他功能连了起来。这个问题让我意识到，PA1 中几个命令虽然是分开实现的，但它们实际上不是孤立功能，而是逐步搭起来的一套调试基础设施。

9.4 表达式求值里最难处理的是单目负号和二元减号的区分

表达式部分真正花时间最多的，不是把数字或运算符切成 token，而是把“看起来一样、语义却不同”的符号区分开。最典型的就是一：它既可能表示减法，也可能表示单目负号。

这个问题一开始没有处理好时，像 `-1+2` 这样的表达式就会算错。原因在于，如果一看到开头的 `-` 就直接把后面整段表达式整体取负，那么等价于把 `-1+2` 解释成 `-(1+2)`，结果自然不对。真正正确的结构应该是先识别出 `-1`，再和后面的 `+2` 按优先级组合。

这个问题最后是分两层解决的：

- 在词法分析阶段，根据前一个 token 是否“能够形成一个值”，把 `-` 区分成二元减号或 `TK_NEG`；

- 在递归求值阶段，先找二元主运算符，只有找不到时才把当前区间当成单目运算符表达式处理。

这样一来，`-1+2`、`--1`、`1-2-3` 这些容易混淆的情况都能按正确结构求值。这也是 PA1 里让我感受最明显的一点：表达式求值看上去只是一个小模块，真正实现起来却已经有明显的“编译器前端”味道，问题不再只是算术，而是先分析结构再计算结果。

9.5 token 缓冲区越界问题不能靠“把数组开大一点”解决

写 `make_token()` 时，另一个比较典型的问题是 token 数组的越界风险。最初如果只是默认输入表达式都不会太长，这个问题很容易被忽略；但只要输入一个足够长的表达式，就有可能把固定长度的 `tokens` 数组写穿。

刚开始最容易想到的办法其实是“把数组开大一点”，但这样只是把问题往后推，并没有真正解决。因为无论数组开成 32、64 还是 128，只要没有边界检查，理论上都还会溢出。

后面做的修改是专门补了一个 `append_token()`，把 token 写入数组前的检查统一放到这个函数里处理：

- token 数量超过上限时直接报错返回；
- 单个 token 太长时直接报错返回；
- `make_token()` 失败后，由 `expr()` 把 `success` 置为 `false`，再由上层命令决定如何提示。

这样之后，非法输入不会再把 monitor 拖进莫名其妙的状态，而是能够比较稳定地报出 `Bad expression` 并回到命令行。这个问题让我意识到，实验里很多 bug 并不来自“主逻辑写错”，而是来自边界条件没有守住。

9.6 监视点一开始的返回值设计不合理，导致总是被误判为触发

监视点部分看起来代码量不算最大，但调试时也踩了一个比较典型的坑：最开始在设计“监视点是否发生变化”的检测函数时，返回值没有设计好，结果只要进入检查流程，就几乎总会被误判成“监视点已触发”。

问题的根源在于，当时把这个函数设计成了返回一个动态分配结果的形式，思路有些绕。实际上，监视点检查最核心的问题只有一个：当前所有监视点里，是否存在至少一个表达式的值发生了变化。对于这个需求，真正合适的返回值其实就是布尔量，而不是复杂的数据结构。

后面把逻辑改成 `check_watchpoints()` 返回 `bool` 之后，整个流程就顺了很多：

- 没有任何表达式变化时返回 `false`；
- 只要某个监视点值变化，或者表达式重新求值失败，就返回 `true`；
- `cpu_exec()` 在每条指令后根据这个返回值决定是否停止执行。

这个修改之后，监视点的行为才稳定下来。这个问题也提醒我，在写代码之前先把“函数到底要表达什么语义”想清楚，往往比先急着堆实现细节更重要。

10. 总结

PA1 中完成的主要任务包括：

- 实现了正确的 `CPU_state` 寄存器结构体；
- 实现了 `si`、`info r`、`x` 等基础调试命令；
- 实现了表达式词法分析、递归求值与 `p` 命令；
- 实现了监视点池管理与监视点触发机制。

更重要的是，PA1 使调试器的工作方式第一次以实现者视角变得具体起来。通过 `si`、`info r` 和 `x`，可以清楚看到 monitor 如何与 CPU 状态和内存接口配合；通过表达式求值，可以看到字符串如何先被切成 token，再按结构求值；通过监视点，则可以看到调试器不仅能“观察状态”，还能“根据状态变化自动停下”。

从指导书的整体安排来看，PA1 的真正目标并不只是“把几个命令补出来”，而是建立起后续实验所依赖的调试基础设施。寄存器组织、命令循环、表达式系统和监视点机制，这些内容在 PA1 中看似分散，实际上共同构成了 NEMU 后续可持续扩展的基础。完成 PA1 之后，“程序如何在计算机上运行”这一问题也不再停留在课本描述上，而是可以从具体代码和具体状态变化中去理解。